

Python NUMPY

What is NumPy?

NumPy is a Python library designed for efficient array operations. It includes powerful tools for working with linear algebra, Fourier transforms, and matrices.

Created in 2005 by Travis Oliphant, NumPy is an open-source project that is free to use. The name **NumPy** stands for **Numerical Python**.

Why Use NumPy?

While Python lists can function as arrays, they are comparatively slow. NumPy provides the **ndarray**, a high-performance array object that can be **up to 50 times faster** than standard Python lists. It also comes with a wide range of functions that simplify array manipulation.

Because of its speed and efficiency, NumPy is widely used in data science and other fields where performance and resource optimization are essential.

Why is NumPy Faster Than Python Lists?

NumPy arrays are stored in a single, continuous block of memory, unlike Python lists. This allows the CPU to access and process data more efficiently—a concept known in computer science as **locality of reference**. This memory layout, combined with optimizations for modern CPU architectures, is the primary reason NumPy outperforms standard Python lists.

What Language Is NumPy Written In?

NumPy is a Python library, but the performance-critical components are implemented in **C** and **C++**, enabling fast numerical computation.

Importing NumPy

After installing NumPy, you can import it into your Python program:

```
import numpy as np  
  
arr = np.array([1, 2, 3, 4, 5])  
  
print(arr)
```

NumPy vs. Python Lists

To understand why NumPy is faster than native Python lists, let's compare the performance of adding two large lists versus adding two NumPy arrays.

Example: Adding Two Lists vs. Adding Two NumPy Arrays

```
import numpy as np

import time

# Python list

size = 1000000

list1 = list(range(size))
list2 = list(range(size))

start = time.time()
result = [x + y for x, y in zip(list1, list2)]
end = time.time()
print("Python list addition time:", end - start)

# NumPy array

arr1 = np.array(list1)
arr2 = np.array(list2)

start = time.time()
result = arr1 + arr2 # Vectorized operation
end = time.time()

print("NumPy array addition time:", end - start)
```

Important:

NumPy is **significantly faster** than Python lists because:

- NumPy operations run in **optimized C code**, avoiding slow Python-level loops.
- NumPy uses **vectorized operations**, processing entire arrays at once.
- Data in NumPy arrays is stored in a **contiguous memory block**, allowing faster access and computation.
- The shape attribute reveals the dimensions of a NumPy array, giving structure to the data.

Creating a NumPy ndarray Object

NumPy is designed for working with arrays. Its core data structure is the **ndarray**. You can create an ndarray using the `array()` function.

Example

```
import numpy as np
arr = np.array([1, 2, 3, 4, 5])
print(arr)
print(type(arr))
print(arr.shape)
```

The built-in Python function `type()` shows that `arr` is of type `numpy.ndarray`.

You can pass a list, tuple, or any array-like object to `array()`, and NumPy will convert it into an ndarray.

Example — Using a Tuple

```
import numpy as np
arr = np.array((1, 2, 3, 4, 5))
print(arr)
```

Dimensions in Arrays

A **nested array** is simply an array that contains other arrays as elements.

0-D Arrays (Scalars)

A 0-D array contains a single value, much like a scalar.

Example

```
import numpy as np
arr = np.array(42)
print(arr)
```

1-D Arrays

A 1-D array contains multiple 0-D elements.
This is the most common type of array.

Example

```
import numpy as np  
arr = np.array([1, 2, 3, 4, 5])  
print(arr)
```

2-D Arrays

A 2-D array contains multiple 1-D arrays.
These are often used to represent matrices or second-order tensors.
NumPy includes a submodule for matrix operations called `numpy.mat`.

Example

```
import numpy as np  
arr = np.array([[1, 2, 3], [4, 5, 6]])  
print(arr)
```

3-D Arrays

A 3-D array contains multiple 2-D arrays.
These are commonly used to represent third-order tensors.

Example

```
import numpy as np  
arr = np.array([[[1, 2, 3], [4, 5, 6]],  
                [[1, 2, 3], [4, 5, 6]]])  
print(arr)
```

Checking the Number of Dimensions

You can check an array's number of dimensions using the `ndim` attribute.

Example

```
import numpy as np

a = np.array(42)

b = np.array([1, 2, 3, 4, 5])

c = np.array([[1, 2, 3], [4, 5, 6]])

d = np.array([[[1, 2, 3], [4, 5, 6]],

               [[1, 2, 3], [4, 5, 6]]])

print(a.ndim)

print(b.ndim)

print(c.ndim)

print(d.ndim)
```

Higher-Dimensional Arrays

NumPy arrays can have any number of dimensions.

You can explicitly set the number of dimensions when creating an array using the `ndmin` parameter.

Example — Creating a 5-D Array

```
import numpy as np

arr = np.array([1, 2, 3, 4], ndmin=5)

print(arr)

print('number of dimensions:', arr.ndim)
```

Creating Arrays from Scratch:

```
np.zeros((3, 3))

np.ones((2, 4))

np.full((2, 2), 7)

np.eye(4)

np.arange(1, 10, 2) # [1, 3, 5, 7, 9] (like range)

np.linspace(0, 1, 5) # [0. 0.25 0.5 0.75 1.] (evenly spaced)
```

Checking Array Properties

```
arr = np.array([[10, 20, 30], [40, 50, 60]])  
print("Shape:", arr.shape)  
print("Size:", arr.size)  
print("Dimensions:", arr.ndim)  
print("Data type:", arr.dtype)
```

Changing Data Types

```
arr = np.array([1, 2, 3], dtype=np.float32)  
arr_int = arr.astype(np.int32) # Convert float to int
```

Reshaping and Flattening Arrays

```
arr = np.array([[1, 2, 3], [4, 5, 6]])  
print(arr.shape)  
new_shape = arr.reshape((3, 2))  
print(new_shape)  
flattened = arr.flatten()
```

Accessing Array Elements in NumPy

- Array indexing refers to accessing individual elements of an array using their index number.
- In NumPy, indexing works the same way as indexing in Python lists.
- Indexing starts at 0:
 - First element → index 0
 - Second element → index 1
 - Third element → index 2, and so on.

Important Points

- Use square brackets [] to access array elements.
- General syntax:
- `array[index]`
- Accessing an index that does not exist will cause an `IndexError`.

Example: Accessing the First Element

```
import numpy as np
```

```
arr = np.array([1, 2, 3, 4])  
print(arr[0])    # Output: 1
```

Explanation

- `arr` contains four elements.
- `arr[0]` accesses the first element, which is 1.

Additional Examples

Purpose	Code	Output
Access 2nd element	<code>arr[1]</code>	2
Access 4th element	<code>arr[3]</code>	4
Access last element	<code>arr[-1]</code>	4
Access second last element	<code>arr[-2]</code>	3

Array Slicing

- Slicing means extracting a portion of an array from a selected range.
- Syntax for slicing:

```
array[start:end]
```

```
array[start:end:step]
```

- Behavior when values are omitted:

Parameter	If Not Provided	Default Value
start	Beginning of array	0
end	End of array	Length of array
step	Step value	1

- Slicing **includes the start index** but **excludes the end index**.

Basic Slicing Examples

Slice from index 1 to 5 (5 not included)

```
import numpy as np
```

```
arr = np.array([1, 2, 3, 4, 5, 6, 7])
```

```
print(arr[1:5]) # Output: [2 3 4 5]
```

Slice from index 4 to the end

```
print(arr[4:]) # Output: [5 6 7]
```

Slice from the beginning to index 4 (4 not included)

```
print(arr[:4]) # Output: [1 2 3 4]
```

Negative Slicing

- Negative indexes count from the end of the array.

Example: Slice from the 3rd last to the 1st last index (end not included):

```
print(arr[-3:-1]) # Output: [5 6]
```

Step in Slicing

- The step value controls the interval between elements.

Every 2nd element from index 1 to index 5

```
print(arr[1:5:2]) # Output: [2 4]
```

Every 2nd element from the entire array

```
print(arr[::-2]) # Output: [1 3 5 7]
```

Slicing 2-D Arrays

Accessing Rows and Columns

- Syntax format:
- `array[row_start:row_end, col_start:col_end]`

Example 1: Slice row with index 1, columns 1 to 3

```
arr = np.array([[1, 2, 3, 4, 5],  
               [6, 7, 8, 9, 10]])
```

```
print(arr[1, 1:4]) # Output: [7 8 9]
```

- Second row has index 1.

Example 2: Select column index 2 from both rows

```
print(arr[0:2, 2]) # Output: [3 8]
```

Example 3: Slice rows 0 to 1 and columns 1 to 3

```
print(arr[0:2, 1:4])
```

Output:

```
# [[2 3 4]
```

```
# [7 8 9]]
```

- This returns a 2-D sub-array.

Slicing in NumPy: View vs Copy

- In NumPy, slicing **returns a view**, not a copy.
- This means the sliced array and the original array share the **same memory location**.
- Any modification made to the slice will also affect the original array.

Difference from Python Lists

- Python list slicing creates a **new, independent copy** of the data.
- NumPy slicing behaves differently by returning a **view** for memory and performance efficiency.

Why Does NumPy Use Views?

1. Memory Efficiency

- Avoids creating unnecessary duplicates of large arrays.
- Saves RAM, especially when working with big datasets.

2. Performance Benefits

- Faster execution because data is not copied.
- Enables efficient manipulation of array segments.

Example: Slice Affecting the Original Array

```
import numpy as np

arr = np.array([10, 20, 30, 40, 50])
slice_arr = arr[1:4]

slice_arr[0] = 200 # modifying the slice
print(arr)        # Output: [10 200 30 40 50]
```

- The original array changes because slice_arr is a view.

Creating an Independent Copy

- If you need a separate array that does not affect the original, use `.copy()`:

```
copy_arr = arr[1:4].copy()
copy_arr[0] = 500
print(arr)    # Original remains unchanged
print(copy_arr) # Independent copy
```

Fancy Indexing and Boolean Masking in NumPy

Fancy Indexing (Selecting Multiple Elements)

- Fancy indexing allows selecting multiple array elements by using a list or array of indices.
- This enables non-sequential access to elements without loops.

Example

```
import numpy as np
```

```
arr = np.array([10, 20, 30, 40, 50])
```

```
idx = [0, 2, 4]    # Indices to select
```

```
print(arr[idx])    # Output: [10 30 50]
```

Key Points

- Access elements using a list of positions.
- Useful for extracting specific, scattered values.
- Does not require slicing ranges; flexible element selection.

Boolean Masking (Filtering Data)

- Boolean masking uses a condition to filter elements.
- The condition returns a boolean array, which is then used to select elements.

Example

```
arr = np.array([10, 20, 30, 40, 50])
```

```
mask = arr > 25    # Condition: filter values greater than 25
```

```
print(arr[mask])    # Output: [30 40 50]
```

Key Points

- Returns elements where the condition is True.
- Efficient for filtering large datasets.
- Commonly used for data cleaning, statistical analysis, and selection based on criteria.

NumPy Data Types

NumPy supports additional data types and represents them using single-character codes:

Code	Type Description
i	integer
b	boolean
u	unsigned integer (no negative values)
f	float
c	complex float
m	timedelta
M	datetime
O	object
S	string
U	unicode string
V	void (fixed memory block)

Checking Data Type of an Array

NumPy arrays have a property `.dtype` which returns the data type of the elements.

Example

```
import numpy as np  
arr = np.array([1, 2, 3, 4])  
print(arr.dtype)  # int32 or int64 (depends on system)
```

Example with Strings

```
arr = np.array(['apple', 'banana', 'cherry'])  
print(arr.dtype)  # <U6 (unicode)
```

Creating Arrays With a Defined Data Type

You can specify a data type when creating an array using the `dtype` parameter.

Example: String Data Type

```
arr = np.array([1, 2, 3, 4], dtype='S')  
print(arr)  
print(arr.dtype)  # string type
```

Example: Fixed Size Integer (4 bytes)

```
arr = np.array([1, 2, 3, 4], dtype='i4')  
print(arr)  
print(arr.dtype)  # int32
```

Value Errors When Type Conversion Fails

If NumPy cannot convert a value to the specified data type, a `ValueError` occurs.

Example of Error

```
arr = np.array(['a', '2', '3'], dtype='i')  # 'a' cannot convert to integer
```

Converting Data Types of Existing Arrays

Use the `.astype()` method to convert an array to a new type.
This method always returns a **copy**, not a view.

Convert float to integer (using string type code)

```
arr = np.array([1.1, 2.1, 3.1])  
newarr = arr.astype('i')  
print(newarr)  
print(newarr.dtype)
```

Convert float to integer (using Python type)

```
newarr = arr.astype(int)  
print(newarr)  
print(newarr.dtype)
```

Convert integer to boolean

```
arr = np.array([1, 0, 3])
```

```
newarr = arr.astype(bool)

print(newarr)

print(newarr.dtype)
```

Iterating Arrays in NumPy

- Iteration means accessing array elements one by one.
- NumPy arrays can be iterated using normal Python for loops.
- For multi-dimensional arrays, iteration happens dimension by dimension.

1. Iterating 1-D Arrays

Iterates through each element directly.

```
import numpy as np

arr = np.array([1, 2, 3])
```

```
for x in arr:

    print(x)
```

2. Iterating 2-D Arrays

Iteration occurs row-by-row.

```
arr = np.array([[1, 2, 3], [4, 5, 6]])
```

```
for x in arr:

    print(x)  # prints each row
```

To access individual scalar values:

```
for x in arr:

    for y in x:

        print(y)
```

3. Iterating 3-D Arrays

Iteration goes through 2-D blocks first.

```
arr = np.array([  
    [[1, 2, 3], [4, 5, 6]],  
    [[7, 8, 9], [10, 11, 12]]  
])
```

```
for x in arr:
```

```
    print(x) # prints each 2-D array
```

Iterating down to scalars:

```
for x in arr:
```

```
    for y in x:
```

```
        for z in y:
```

```
            print(z)
```

4. Iterating with np.nditer()

A more advanced iteration tool that simplifies looping through every element.

Basic usage

```
for x in np.nditer(arr):
```

```
    print(x)
```

Changing Data Type While Iterating

Use `op_dtypes` and `buffered` flag:

```
arr = np.array([1, 2, 3])
```

```
for x in np.nditer(arr, flags=['buffered'], op_dtypes=['S']):
```

```
    print(x) # prints as string type
```

Iterating with Step Size (Skipping Values)

```
arr = np.array([[1, 2, 3, 4], [5, 6, 7, 8]])
```

```
for x in np.nditer(arr[:, ::2]):  
    print(x)  # skips every other column
```

5. Enumerated Iteration: ndenumerate()

Returns both index and value.

1-D Example

```
arr = np.array([1, 2, 3])
```

```
for idx, x in np.ndenumerate(arr):  
    print(idx, x)
```

2-D Example

```
arr = np.array([[1, 2, 3], [4, 5, 6]])
```

```
for idx, x in np.ndenumerate(arr):  
    print(idx, x)
```

Joining NumPy Arrays

Overview

- Joining means combining multiple arrays into a single array.
- Unlike SQL joins (based on keys), NumPy joins arrays based on axes.
- The primary function for joining arrays is `np.concatenate()`.

Using `concatenate()`

- Syntax:
`np.concatenate((array1, array2), axis=0)`

- If axis is not specified, it defaults to 0.

Example: Joining 1-D Arrays

```
import numpy as np
```

```
arr1 = np.array([1, 2, 3])
```

```
arr2 = np.array([4, 5, 6])
```

```
arr = np.concatenate((arr1, arr2))
```

```
print(arr)    # Output: [1 2 3 4 5 6]
```

Example: Joining 2-D Arrays Along Columns (axis = 1)

```
arr1 = np.array([[1, 2], [3, 4]])
```

```
arr2 = np.array([[5, 6], [7, 8]])
```

```
arr = np.concatenate((arr1, arr2), axis=1)
```

```
print(arr)
```

```
# Output:
```

```
# [[1 2 5 6]
```

```
# [3 4 7 8]]
```

Stacking Arrays (Creating a New Axis)

Stacking is similar to concatenation, but a **new dimension** is added.

Using stack()

```
arr1 = np.array([1, 2, 3])
```

```
arr2 = np.array([4, 5, 6])
```

```
arr = np.stack((arr1, arr2), axis=1)
```

```
print(arr)
```

```
# Output:
```

```
# [[1 4]
# [2 5]
# [3 6]]
```

Helper Functions for Stacking

1. `hstack()` → Horizontal stacking (along rows)

```
arr = np.hstack((arr1, arr2))
print(arr) # Output: [1 2 3 4 5 6]
```

2. `vstack()` → Vertical stacking (along columns)

```
arr = np.vstack((arr1, arr2))
print(arr)
# Output:
```

```
# [[1 2 3]
# [4 5 6]]
```

3. `dstack()` → Depth stacking (along height)

```
arr = np.dstack((arr1, arr2))
print(arr)
# Output:
# [[[1 4]
# [2 5]
# [3 6]]]
```

Splitting NumPy Arrays

Overview

- Splitting is the reverse of joining.
- Joining combines multiple arrays → one array
- Splitting breaks one array → multiple arrays
- Primary function: `np.array_split()`

Using `array_split()`

Syntax

```
np.array_split(array, number_of_splits)
```

Example: Split into 3 Parts

```
import numpy as np
```

```
arr = np.array([1, 2, 3, 4, 5, 6])
```

```
newarr = np.array_split(arr, 3)
```

```
print(newarr)
```

```
# Output: [array([1,2]), array([3,4]), array([5,6])]
```

The return value is a **list of arrays**.

Handling Uneven Splits

If the array has fewer elements than needed for perfect equal splits, NumPy adjusts automatically from the end.

```
arr = np.array([1, 2, 3, 4, 5, 6])
```

```
newarr = np.array_split(arr, 4)
```

```
print(newarr)
```

```
# Output: [array([1,2]), array([3,4]), array([5]), array([6])]
```

`split()` vs `array_split()`

Function	Handles Uneven Splits?	Behavior
<code>split()</code>	No	Will raise an error
<code>array_split()</code>	Yes	Adjusts automatically

Accessing Split Arrays

```
newarr = np.array_split(arr, 3)

print(newarr[0])

print(newarr[1])

print(newarr[2])
```

Splitting 2-D Arrays

Split into 3 sub-arrays (row-wise)

```
arr = np.array([[1,2],[3,4],[5,6],[7,8],[9,10],[11,12]])

newarr = np.array_split(arr, 3)

print(newarr)
```

Returns three 2-D arrays.

Split 2-D Array into 3 parts (along columns → axis=1)

```
arr = np.array([[1,2,3],[4,5,6],[7,8,9],[10,11,12],
               [13,14,15],[16,17,18]])

newarr = np.array_split(arr, 3, axis=1)

print(newarr)
```

Alternate Splitting Methods

Function	Splits Along	Opposite Of
hsplit()	Columns (axis=1)	hstack()
vsplit()	Rows (axis=0)	vstack()
dsplit()	Depth (axis=2)	dstack()

Example: `hsplit()`

```
arr = np.array([[1,2,3],[4,5,6],[7,8,9],[10,11,12],  
               [13,14,15],[16,17,18]])
```

```
newarr = np.hsplit(arr, 3)
```

```
print(newarr)
```

Searching Arrays in NumPy

NumPy offers efficient methods for searching arrays:

- **`np.where()`** → find indexes where a condition is true.
- **`np.searchsorted()`** → find the position where a value should be inserted to maintain sorted order (uses binary search).

1. Searching Arrays with `where()`

What it does

`np.where()` allows you to search an array for values that match a condition. It returns a **tuple of arrays**, containing the indexes where the condition is true.

Example — Find indexes where the value equals 4

```
import numpy as np
```

```
arr = np.array([1, 2, 3, 4, 5, 4, 4])
```

```
x = np.where(arr == 4)
```

```
print(x)
```

Output:

```
(array([3, 5, 6]),)
```

Explanation:

- The value **4** appears at **index 3, 5, and 6**.

Example — Find indexes of even numbers

```
import numpy as np
```

```
arr = np.array([1, 2, 3, 4, 5, 6, 7, 8])
```

```
x = np.where(arr % 2 == 0)
```

```
print(x)
```

- Condition: $\text{arr \% 2} == 0 \rightarrow \text{True for even numbers.}$

Example — Find indexes of odd numbers

```
import numpy as np
```

```
arr = np.array([1, 2, 3, 4, 5, 6, 7, 8])
```

```
x = np.where(arr % 2 == 1)
```

```
print(x)
```

- Condition: $\text{arr \% 2} == 1 \rightarrow \text{True for odd numbers.}$

2. Searching Sorted Arrays with `searchsorted()`

What it does

- Performs a **binary search** on a sorted array.
- Returns the **index where a value should be inserted** to maintain sorted order.
- Much faster for large sorted arrays.
- Works only correctly on **sorted arrays**.

- Default behavior searches **from the left**.
- Can specify side='right' to search from the right.

Example — Insert position for value 7

```
import numpy as np
```

```
arr = np.array([6, 7, 8, 9])
```

```
x = np.searchsorted(arr, 7)
```

```
print(x)
```

Explanation:

- Search starts from the left.
- 7 should be inserted at **index 1** to keep order.

Search From the Right Side

Setting side='right' returns the index where the value should be inserted on the right side of any existing matches.

Example 2 — Insert position for value 7 (right side)

```
import numpy as np
```

```
arr = np.array([6, 7, 8, 9])
```

```
x = np.searchsorted(arr, 7, side='right')
```

```
print(x)
```

Explanation:

- Search starts from the right.

- 7 should be inserted at **index 2** to preserve order.

3. Searching for Multiple Values

You can pass multiple values to `searchsorted()` by using a list/array.

Example — Find insert positions for values 2, 4, and 6

```
import numpy as np
```

```
arr = np.array([1, 3, 5, 7])
```

```
x = np.searchsorted(arr, [2, 4, 6])
```

```
print(x)
```

Output:

```
[1 2 3]
```

Explanation:

- 2 → insert at index **1**
- 4 → insert at index **2**
- 6 → insert at index **3**

These positions maintain the sorted order of the original array.

Sorting Arrays in NumPy

Sorting is the process of arranging elements in an ordered sequence. This order can be:

- **Numeric** (e.g., 0 → 1 → 2 → 3)
- **Alphabetical** (e.g., apple → banana → cherry)
- **Boolean** (False → True)
- **Ascending or descending** order

NumPy provides the **`np.sort()`** function, which sorts elements in an array and **returns a new sorted copy**, leaving the original array unchanged.

Sorting a 1-D Array

- `np.sort()` sorts elements in ascending order.
- It does **not modify** the original array.
- Works on any data type: numbers, strings, booleans.

Example — Sorting a numeric array

```
import numpy as np
```

```
arr = np.array([3, 2, 0, 1])
```

```
print(np.sort(arr))
```

Output:

```
[0 1 2 3]
```

Explanation:

The array is sorted from smallest → largest.

Sorting Strings

NumPy sorts strings alphabetically (lexicographical order).

Example — Sorting strings

```
import numpy as np
```

```
arr = np.array(['banana', 'cherry', 'apple'])
```

```
print(np.sort(arr))
```

Output:

```
['apple' 'banana' 'cherry']
```

Sorting Boolean Arrays

Boolean values are treated as:

- **False = 0**
- **True = 1**

So sorting places False before True.

Example — Sorting booleans

```
import numpy as np
```

```
arr = np.array([True, False, True])
```

```
print(np.sort(arr))
```

Output:

```
[False True True]
```

Sorting a 2-D Array

When sorting a 2-D array, NumPy sorts **each row independently**.

Key Notes

- Sorting is **row-wise**.
- Every row becomes a sorted 1-D array.
- The shape of the array remains the same.

Example — Sorting a 2-D array

```
import numpy as np
```

```
arr = np.array([[3, 2, 4], [5, 0, 1]])
```

```
print(np.sort(arr))
```

Output:

```
[[2 3 4]
```

[0 1 5]]

Explanation:

- First row: 3, 2, 4 $\rightarrow$ 2, 3, 4
- Second row: 5, 0, 1 $\rightarrow$ 0, 1, 5

